

E-Commerce Product Sorting System Using Merge Sort (Divide and Conquer Approach)

1. Description of the Real-World Problem

Modern e-commerce platforms such as Amazon, Flipkart, and Myntra contain millions of products. Customers expect products to be displayed quickly after applying filters such as **price**, **customer rating**, or **discount**.

As the number of products increases, sorting them efficiently becomes a major challenge. The sorting algorithm should not only be fast but should also preserve the order of products having the same value (stable sorting). Therefore, an efficient sorting technique is required to handle large datasets with minimal delay.

2. Problem Statement

Design a product sorting system for an e-commerce platform that sorts millions of products according to their **price** (ascending order). If two products have the same price, their original order should remain unchanged.

The system should:

- Handle very large datasets efficiently.
- Use the Divide and Conquer strategy.
- Provide consistent performance.
- Maintain stability during sorting.
- Reduce response time for users.

3. Algorithm Used

Merge Sort

Merge Sort is a **Divide and Conquer** algorithm.

Working Principle

Step 1: Divide the array into two equal halves.

Step 2: Continue dividing until each sub-array contains only one element.

Step 3: Merge the sub-arrays in sorted order.

Step 4: Continue merging until the complete array becomes sorted.

Divide and Conquer Steps

- **Divide:** Split the product list into smaller parts.
- **Conquer:** Sort each part recursively.
- **Combine:** Merge the sorted parts into one sorted list.

4. Justification for Choosing the Algorithm (Why Merge Sort?)

Merge Sort is suitable because:

- It guarantees **$O(n \log n)$** time complexity even for large datasets.
- It is a **stable sorting algorithm**, preserving the order of products with equal prices.
- Performance remains consistent regardless of input.
- Suitable for external sorting where data cannot fit entirely into memory.
- Can be parallelized easily in distributed systems.

Hence, Merge Sort is widely used in large-scale data processing applications.

5. Complexity Analysis

Case	Time Complexity
Best case	$O(n \log n)$
Average case	$O(n \log n)$
Worst case	$O(n \log n)$

6. Manual Execution (Step-by-Step Process)

Example Product Prices

45, 20, 60, 10, 35, 80, 25, 50

Step 1: Divide

- 1.) 45 20 60 10 | 35 80 25 50
 - 2.) 45 20 | 60 10 | 35 80 | 25 50
 - 3.) 45 | 20 | 60 | 10 | 35 | 80 | 25 | 50
- Each sub-array now contains one element.

Step 2: Merge

Merge 45 + 20 → 20 45

Merge 60 + 10 → 10 60

Merge 35 + 80 → 35 80

Merge 25 + 50 → 25 50

Step 3: Merge Again

20 45 + 10 60 → 10 20 45 60

35 80 + 25 50 → 25 35 50 80

Step 4: Final Merge

10 20 45 60 + 25 35 50 80 → 10 20 25 35 45 50 60 80

Final Sorted Product Prices

10, 20, 25, 35, 45, 50, 60, 80

7. Applications

Merge Sort is widely used in:

- E-commerce product sorting
- Database management systems
- External file sorting
- Search engine indexing
- Banking transaction processing
- Big data analytics
- Distributed computing
- Cloud-based applications

8. Advantages

- Efficient for large datasets.
- Stable sorting algorithm.
- Guaranteed $O(n \log n)$ performance.
- Suitable for linked lists.

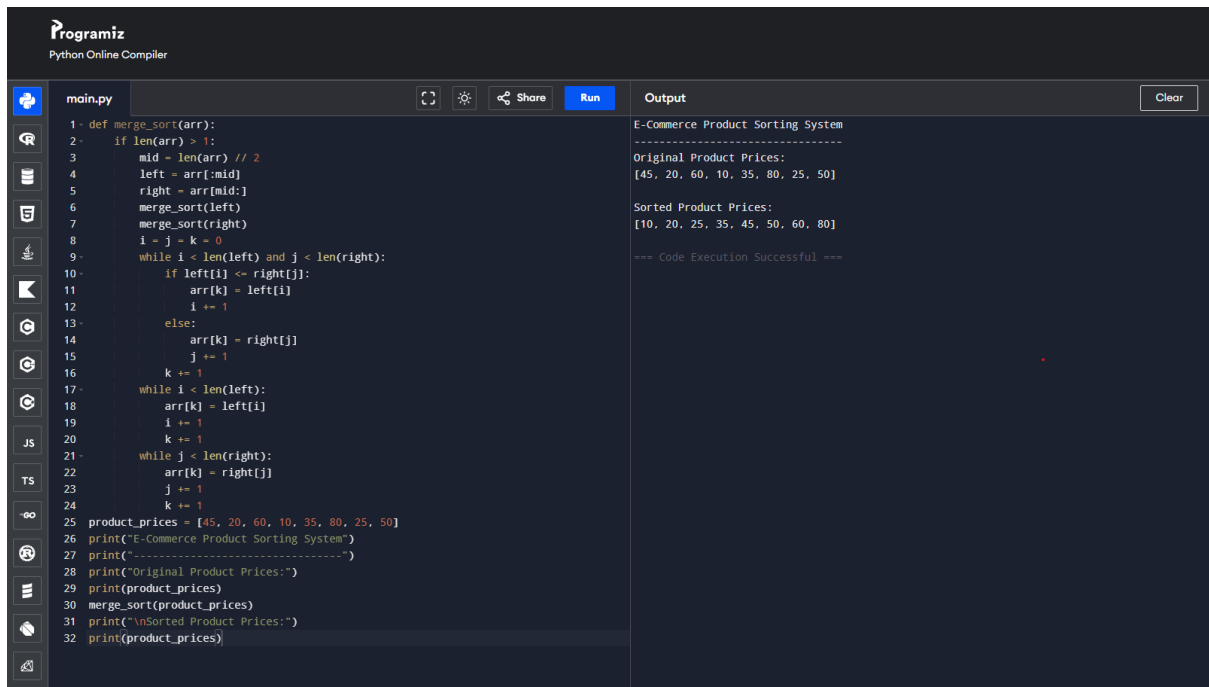
- Easy to implement using recursion.
- Works well for external sorting.
- Can be parallelized.

9. Limitations

- Requires additional memory $O(n)$.
- Slower than Quick Sort for small datasets.
- Recursive implementation consumes stack space.
- Not an in-place sorting algorithm.

10. Comparison with Other Algorithms

Algorithm	Best	Average	Worst	Stable	Suitable for Large Data
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	Excellent
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	No
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	No
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	Small datasets
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No	Yes



The screenshot displays the Programiz Python Online Compiler interface. On the left, a sidebar contains icons for various programming languages and tools. The main editor area shows a Python file named 'main.py' with a Merge Sort implementation. The code defines a 'merge_sort' function that recursively splits an array into halves and then merges them back in sorted order. It uses a 'product_prices' list as input. The output panel on the right shows the execution results, including the title 'E-Commerce Product Sorting System', the original product prices, and the sorted product prices. A message at the bottom of the output panel states '=== Code Execution Successful ==='.

```
1- def merge_sort(arr):
2-     if len(arr) > 1:
3-         mid = len(arr) // 2
4-         left = arr[:mid]
5-         right = arr[mid:]
6-         merge_sort(left)
7-         merge_sort(right)
8-         i = j = k = 0
9-         while i < len(left) and j < len(right):
10-            if left[i] <= right[j]:
11-                arr[k] = left[i]
12-                i += 1
13-            else:
14-                arr[k] = right[j]
15-                j += 1
16-            k += 1
17-         while i < len(left):
18-             arr[k] = left[i]
19-             i += 1
20-             k += 1
21-         while j < len(right):
22-             arr[k] = right[j]
23-             j += 1
24-             k += 1
25- product_prices = [45, 20, 60, 10, 35, 80, 25, 50]
26- print("E-Commerce Product Sorting System")
27- print("-----")
28- print("Original Product Prices:")
29- print(product_prices)
30- merge_sort(product_prices)
31- print("\nSorted Product Prices:")
32- print(product_prices)
```

E-Commerce Product Sorting System

Original Product Prices:
[45, 20, 60, 10, 35, 80, 25, 50]

Sorted Product Prices:
[10, 20, 25, 35, 45, 50, 60, 80]

=== Code Execution Successful ===

11. Conclusion

Merge Sort is an efficient and reliable sorting algorithm for large-scale e-commerce applications. By using the **Divide and Conquer** strategy, it consistently achieves **$O(n \log n)$** time complexity while maintaining the stability of equal-priced products. Although it requires additional memory, its predictable performance and scalability make it an excellent choice for sorting millions of products in modern online shopping platforms, ensuring faster search results and an improved user experience.